Working Guidelines

What NOT to do (anti-pattern): do not use midrashim or classical Torah commentaries.
What TO do: read the text critically and meticulously, applying the methodologies of biblical Hebrew linguistics and the methodologies of biblical criticism.

At any given stage, we will focus on one or more segments (commentary files) that I will specify, whose commentary needs to be produced or improved.
To give a quality analysis - read the relevant segments - and perhaps also a few segments before and after, for context.

Quality Tests

The "Reverse-Engineering" Test

The strongest test of the commentary's quality: imagine a writer who must craft a text that works on two layers at once - an overt mythological story, and a hidden cultural-sociological story.
And ask: given both constraints - would the writer choose *exactly these words*?

- "Yes - exactly these words": a strong interpretation.
- "The word is odd or superfluous in the overt story - but called for in the hidden story": this is the strongest signal.
- "The word is natural in the overt story - but the hidden reading requires a stretch": this is a problem in the interpretation - and it must be stated explicitly in the body of the commentary: that the reading is strained, and where.
- "The word fits many hidden readings - and nothing decides between them": the proposed reading is a guess, which does not meet our standard - and its place is inside `<מדרש>`.

This test complements the degrees-of-freedom test: degrees of freedom check whether the dictionary *constrains* the reading, and reverse engineering checks whether the text is *fitted* to it. (This is a working tool - not a claim about the identity of the writer or their intentions.)

Suspicious Reading

Every word in the text is a potential dictionary carrier: don't let words pass as "connective tissue" or as "stylistic variation".
Small words like `עַל` (upon), `כֹּל` (all), and `ל-` (the prepositional lamed: to/for) often carry great weight in the hidden layer.
Grammatical anomalies - an unexpected gender, an unusual verb form, a surprising word order - are signals, not scribal errors.
But first check that the anomaly is not a systematic phenomenon of biblical Hebrew.

Ground the Abstract

Cultural-sociological processes are abstract by nature - and the commentary must make them tangible: analogies, "imagine that..." scenarios, or modern parallels.
That way the reader *feels* what the cultural transition looked like - and doesn't only understand it logically.
If an interpretation cannot be grounded in something tangible - it may be too vague.

Cross-References Between Segments

Many concepts are not in the dictionary - they are established for the first time within the commentary segments themselves: archetypes (נחש, קין, הבל), characters and places, and recurring verbs (ידע, הרה, ילד, נתן, לקח...).
When encountering such a concept:

1. Search for where it was first interpreted - using `./scripts/hebrew-grep` (see below).
2. Read that segment - and understand the meaning that was established there.
3. Apply that meaning consistently: do not re-derive it from scratch, and do not deviate from it without justification.
4. Add a reference for the reader - for example: "כמו שראינו בבראשית ג:א, הנחש הוא..." ("As we saw in Genesis 3:1, the serpent is...").

When a concept will be developed in a later segment - add a forward reference. For example: "נושא זה יתברר לעומק בסיפור קין והבל (בראשית ד)" ("This topic will be clarified in depth in the story of Cain and Abel (Genesis 4)").

Commentary Files

Because the commentary is long, I split it into files, so that only the relevant parts need to be loaded.
The files are under the folders:

- `./פירוש/1-בראשית/`
- `./פירוש/2-שמות/`
- `./פירוש/3-ויקרא/`
- `./פירוש/4-במדבר/`
- `./פירוש/5-דברים/`

File names start with a non-sequential ordinal number (usually in jumps of 10) that sorts the files alphabetically.
For example, the name of the first file is `./פירוש/1-בראשית/1010-בראשית-א_א-ב_ג-שבע_ימי_הבריאה.rtl.md`

All the segment files already exist, and cover every verse in all five books: each file contains the biblical text - **but only some of the files contain a completed commentary**.

The Format of the Commentary Files

The commentary files are in Markdown format.

- A line that quotes a verse starts with ">" - for example:
  > בראשית א א: בְּרֵאשִׁית בָּרָא אֱלֹהִים אֵת הַשָּׁמַיִם וְאֵת הָאָרֶץ.
- Biblical quotations are marked with backquotes - for example `אֵת הַשָּׁמַיִם וְאֵת הָאָרֶץ`.

- "Pseudo HTML Tags" are used to delimit special paragraphs:
  - `<הקבלה-היסטורית/>` ... `<הקבלה-היסטורית>`
    A parenthetical essay that dresses the story onto a specific history.
  - `<עיון/>` ... `<עיון>`
    A parenthetical essay that goes deeper into a relevant topic - "for advanced readers".
  - `<מדרש/>` ... `<מדרש>`
    A parenthetical essay containing an interpretation that is not sufficiently "constrained" by the text, that feels like a guess, or that is simply unconvincing: critical readers are invited to skip it.
  - `<ניתוח-לשוני ביטוי="...expression from the verse...">` ... `</ניתוח-לשוני>`
    A parenthetical essay with a linguistic analysis of a word that appeared in the verse.
  - `<הצעת-קלוד/>` ... `<הצעת-קלוד>`
    This is your way to add suggestions without changing the existing text.
    Very important: before editing existing text (that is, before writing a `<הצעת-קלוד>`) - make sure to read the guidelines in the file `.claude/הנחיות-לעריכת-קבצי-פירוש.rtl.md` !
- A line that starts with "TODO:" highlights a problematic issue that requires future attention.
- All other lines - regular lines - are commentary text.

Linguistic Analyses

Biblical linguistics is an important area of expertise. You have excellent linguistic ability, but deep linguistic analysis needs to be performed with other tools that you don't have access to.
The translation tends to give words in the text a non-intuitive meaning, but a meaning that is well anchored in the essence of the word: in such cases, the commentary will contain a reference to a linguistic-analysis file - under the folder `./ניתוחים-לשוניים/`

Root analysis is a central tool of the commentary - especially for the names of people and places.
When the root is transparent and the meaning fits the dictionary - it can be applied directly in the commentary. When the root is opaque or contested - mark it with `TODO:`.

Important: if you find a biblical word whose deep meaning needs to be fully extracted in order to understand the depth of a verse - please ask me to conduct linguistic research on the word - and to add a reference to the linguistic-analysis file from within the commentary.

Technical Instructions for Working With Hebrew Files

Searching Hebrew Text: `./scripts/hebrew-grep`

Standard grep cannot search Hebrew in this project — niqqud (vowel marks like בְּרֵאשִׁית) and non-Hebrew characters (backticks, markdown) break pattern matching. Always use `./scripts/hebrew-grep` instead.

How it works: Before matching, every line is normalized — niqqud is stripped and all non-Hebrew sequences collapse to a single space. The search sees only space-delimited bare Hebrew words. Spaces are added at line boundaries, so `' נחש '` matches the whole word.

Usage: `./scripts/hebrew-grep <JS RegExp> <files-or-folders...>`
Folders are searched recursively. Output is YAML-like: file path, then matching lines with line numbers and verse references.

Examples:

```
# Find exact word אדמה across all commentary
./scripts/hebrew-grep ' פירוש ' אדמה/

# Find the word תרבות with any prefix in a specific file (will also find והתרבות
./scripts/hebrew-grep 'בראשית-1010/בראשית-א_א-ב_ג-שבע_ימי_הבריאה/1-פירוש ' תרבות

# Find any word with "נ" followed by "ת" followed by "ש" in two files
./scripts/hebrew-grep 'בראשית-1020/בראשית-ב_ד-ב_י-גן_עדן/1-פירוש ' ש*[ ^]ת*[ ^]נ
```

Editing Hebrew Files: Niqqud and Combining-Mark Order

The Edit tool can fail with `String to replace not found` on Hebrew text that looks character-for-character identical to what Read returned. The cause is **Unicode combining-mark order**: niqqud (qamatz ָ, dagesh ּ, etc.) are combining marks, and the same visible word can be encoded with marks in different byte orders (e.g., letter+qamatz+dagesh vs. letter+dagesh+qamatz). Edit does exact byte matching and rejects strings that *look* right but differ in mark order. The error message is unhelpfully generic.

If Edit fails on Hebrew text with niqqud — do NOT retry with a re-typed `old_string`. Re-typing reproduces the same wrong byte order. Switch to Python via Bash:

1. Locate the target block using **niqqud-free anchors** — ASCII or unmarked Hebrew (e.g., `'- כל` `'יכל בהמה ובהמה״`, `'=.'`). These match reliably regardless of mark order.
2. Extract the block verbatim from the file with `content[start:end]` — don't retype the niqqud-bearing portion.
3. Build the replacement (new niqqud you write is fine — its mark order needn't match anything pre-existing).
4. `content.replace(old_block, new_block, 1)` and write back.

```
path = '...'
with open(path, 'r', encoding='utf-8') as f:
    content = f.read()
start = content.find('<niqqud-free start anchor>')
end_anchor = '<niqqud-free end anchor>'
end = content.find(end_anchor) + len(end_anchor)
old_block = content[start:end]
new_block = """..."""
assert content.count(old_block) == 1
content = content.replace(old_block, new_block, 1)
with open(path, 'w', encoding='utf-8') as f:
    f.write(content)
```

Diagnostic when two Hebrew strings look identical but don't match — find the first byte-level divergence:

```
for i, (a, b) in enumerate(zip(actual, target)):
    if a != b:
        print(f"Diff at offset {i}: file={hex(ord(a))}, target={hex(ord(b))}")
        break
```

Restrictions on Modifying Existing Text

Important! Do not modify existing text without the user's approval.
On the other hand - you may always add/edit/delete `<הצעת-קלוד>` sections: that is exactly their purpose.